



A Full-Process Digital Method for Recursive Geometric Structure and Machining Parameter Generation Targeting the Reutilization of Curved Timber

Shenzhou Dai¹[0009-0004-6929-4817], Sihang Liu², Haojun Wang³, Yulin Chen⁴, and Zixiong Lin¹

¹ University of California, Berkeley
shenzhou_dai@berkeley.edu

² The Chinese University of Hong Kong

³ Columbia University

⁴ Carnegie Mellon University

Abstract. The commercial wood processing industry, driven by North American construction, generates a substantial number of by-products annually, including branches, bark, wood shavings, and sawdust. Building upon previous practices in reusing wood by-products, this study focuses on optimizing the utilization of curved timber generated during the logging process. A computational method is proposed to support the fabrication of non-standard wooden products using robotic arms or collaborative robots for future applications. This method facilitates the generation and visualization of structural schemes and construction parameters for the primary framework. The approach relies on GHPython components and is implemented through three key stages: a three-dimensional posture brute-force search based on fixed angle increments, structural combination solving using the SymPy computational library, and node visualization and generation on the Grasshopper platform. These steps enable the acquisition of manufacturing parameters for a stable recursive spatial truss structure while maximizing the utilization of raw materials. The study significantly enhances computational efficiency and validates the structural reliability of the generated framework through finite element analysis.

Keywords: Computational Design, Non-standard Timber Construction, Digital Fabrication, Circular Economy, Finite Element Analysis.

1 Introduction

Since the 17th century, wood has been a primary material in North American construction, widely used in residential and light commercial buildings. Due to its affordability and low construction costs, wood has remained highly favored in the construction industry. In recent years, the rise of the mass timber construction trend has further advanced the development of wooden structures [1]. Driven by supply and demand



dynamics, North America's logging and wood processing industries produce hundreds of millions of cubic meters of commercial timber annually [2].

The process of transforming commercial timber from harvesting to finished products can generally be divided into three stages: harvesting, processing, and packaging. However, due to the curvature and dimensions of branches often failing to meet the processing standards for commercial timber, trees are typically pruned on-site after harvesting, generating a substantial number of by-products [3]. During subsequent factory processing, additional by-products such as bark, wood chunks, and wood shavings are further produced. This process ultimately results in a low Lumber Recovery Factor (LRF), which refers to the proportion of raw timber, measured by volume or weight, converted into usable wood products. Up to 50% of the timber is wasted during production [4].

While a small portion of branches is coarsely processed into landscaping materials or household fuel, which is often accompanied by high carbon emissions, a significant amount of wood waste remains difficult to degrade naturally due to the presence of lignin and cellulose [5]. These by-products not only contribute to resource wastage but also pose potential environmental threats.

The use of naturally shaped timber for constructing shelters by humans can be traced back to approximately 500,000 years ago [6]. These structures, which may now seem "informal," provided safe and reliable spaces for human activities throughout history. In the current context of environmental crises coexisting with rapid technological advancements, leveraging the advantages of machines to create low-cost and environmentally friendly products aimed at achieving carbon reduction goals is achievable. Due to their resistance to natural degradation, wood waste demonstrates potential as an affordable and durable structural material. Meanwhile, robotic arms and collaborative robots, with their high efficiency in repetitive and programmatic operations, offer possibilities for achieving high-efficiency production. Against this backdrop, this study focuses on a digital method oriented toward the structural application of non-standard materials, such as naturally shaped timber. The proposed method aims to facilitate efficient and sustainable material reuse and the production of structural systems.

This study builds upon prior research on the reutilization of by-products generated throughout the production cycle of commercial timber. In previous studies, we explored the reuse of four types of by-products, including curved timber (mainly branches) from the logging stage, alongside bark, wood shavings, and thin wood sheets. These by-products were effectively utilized to construct physical structural products, which were subsequently showcased in practical exhibitions. Among them, wood shavings have been shown to be processed into flexible, lightweight fabric-like structures using programmatic methods, highlighting their potential for practical applications. However, the practical application of curved timber remains challenging due to its inherent material complexity.

First, the complex curvature of curved timber makes constructing stable structures extremely time-consuming. Specifically, it requires repeated manual trials and adjustments in 3D design software to create complete structural assemblies after 3D scanning of the timber. Second, the considerable variation in the length of raw timber necessitates meticulous calculation and precise segmentation, resulting in a substantial workload.



Finally, inaccuracies arising from manual measurements and assembly pose difficulties in accurately designing and fabricating structural joints. In previous practice, we were forced to rely on high-tolerance materials, such as flexible ropes, to connect components. While these flexible joints alleviate inaccuracies to some extent, they compromise structural stability and load-bearing capacity.

To address the challenges of structural generation using curved timber, this study proposes a method based on programming logic. This method enables end-to-end digitization, from parameter collection to result visualization and manufacturing parameter generation, ensuring high automation and efficiency.

This method utilizes GHPython components and the Grasshopper platform to streamline computations and visualize results. By integrating a brute-force search algorithm with complex equation solving based on the SymPy library, it effectively addresses the challenges of parameter collection and structural generation for curved timber. The approach outputs a recursive spatial truss structure assembly based on collected data, automates the generation of spherical nodes and connecting components, and achieves design visualization as well as the output of manufacturing parameters. Finite Element Analysis (FEA) has confirmed the stability and mechanical performance of the resulting structures, demonstrating their feasibility for practical applications.

In recent years, with the rapid advancement of computer technology and intelligent machinery, research on utilizing digital methods to develop wood by-products has garnered increasing attention and yielded numerous innovative outcomes. Monier et al. developed a method based on curved timber to generate enclosing structures according to given freeform surfaces. This method generates two-dimensional truss-like structures in Grasshopper, based on the timber's axis, which conform to the target surface to form a stable cylindrical arch [7]. Mollica et al., in the Wood Chip Barn project, integrated methods based on 3D scanning, Rhinoceros, and Grasshopper to virtually process curved timber and generate machining parameters. Ultimately, they utilized robotic arms to fabricate connection nodes and successfully constructed the designed structure [8]. Reisach et al. focused on the reutilization of waste wood chunks. Their approach, also based on Rhinoceros and Grasshopper, involved the virtual morphological design and structural computation of actual materials. By utilizing specially designed connection components and CNC machining techniques, they constructed the designed structure with high precision [9]. Wang utilized scripting to iterate through a collected database of curved timber, enabling the computer to generate multiple design options for curved wall structures composed of multilayered, horizontally arranged timber components [10]. Although these studies have provided significant inspiration for our research, several gaps remain in the current body of knowledge. First, there is a lack of standardized, automated methods for extracting and controlling timber grid axis accuracy in the workflow of processing 3D scanning results. Second, most related studies focus on assembling point-based objects into linear structures or constructing surfaces with linear components, while research directly using linear components to form spatial structures is relatively scarce. Finally, many studies continue to depend on complex processing steps for raw timber, which substantially reduces manufacturing efficiency.



This study proposes a series of innovative solutions through automated scripting, including the exploration of new structural composition logics, the improvement of component connection methods, and the effective utilization of advanced computational capabilities, aiming to address the challenges.

2 Methodology

2.1 Framework Overview

The proposed framework integrates computational and visualization tools to address the challenges of structural generation using curved timber. It is implemented through GHPython components and the Grasshopper platform, enabling a streamlined workflow for parameter collection, structural generation, result visualization, and manufacturing parameter output.

As illustrated in Figure 1, the workflow of this method is divided into three stages. In the first stage, a three-dimensional posture brute-force search based on fixed-angle increments is employed, significantly enhancing the efficiency of parameter collection for component axes. In the second stage, the method integrates the SymPy computation library to solve geometric structures based on the collected parameters, constructing an abstract recursive spatial truss framework with tetrahedrons as the fundamental units. In the third stage, the materialization of the abstract structure enables the automatic generation of wooden spherical nodes and connecting rods, visualizing the design and outputting the physical parameters required for manufacturing.

The FEA method is employed to simulate stress performance of the generated structures under standard loads, with multiple samples analyzed to evaluate their mechanical performance.

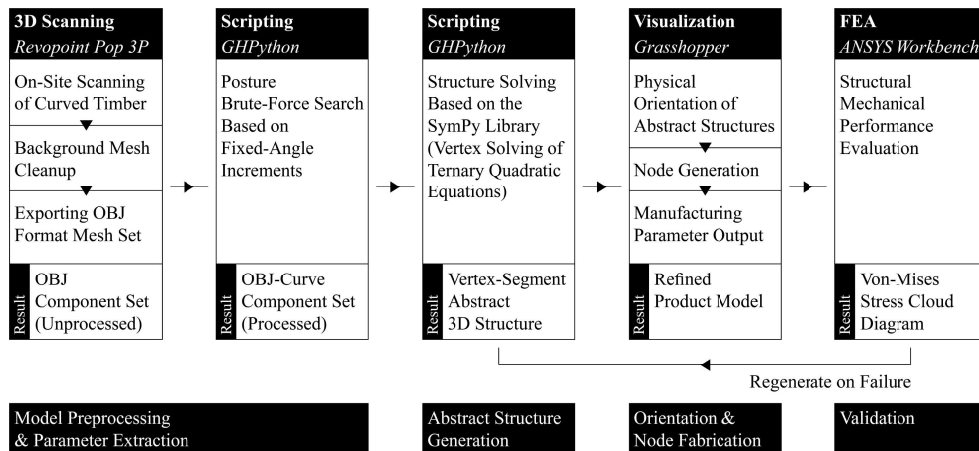


Fig. 1. A flowchart illustrating the framework of this study. Dark sections indicate primary methods and stages, while light sections show specific operations.

2.2 Model Preprocessing and Parameter Extraction

This is the first stage of the research workflow, aimed at digitizing the physical properties of raw timber. This stage comprises a 3D scanning section and a scripting section, focusing on using automated methods to enhance the efficiency of parameter extraction to meet the extensive data collection demands of practical applications.

3D scanning was conducted on-site at the timber harvesting location. Using a handheld 3D scanner, we collected 30 branch models with lengths of up to 1500 millimeters. These timber pieces lacked significant branching and maintained their original orientations during scanning. Subsequently, the scanner's accompanying software was used to automatically remove redundant or floating artifacts. Multiple timber models were exported as individual OBJ objects into Rhinoceros, forming an unprocessed raw model set.

Due to the non-standardized shapes and complex postures of the timber models, the traditional method of finding axes by manually slicing the model presents several challenges when dealing with large quantities of random models. These challenges primarily include difficulties in determining slice direction, controlling slice density, and locating start and end points.

To address the inefficiency caused by uncertainties in slice direction, this study developed a posture brute-force search method based on fixed angle increments, as illustrated in Figure 2.

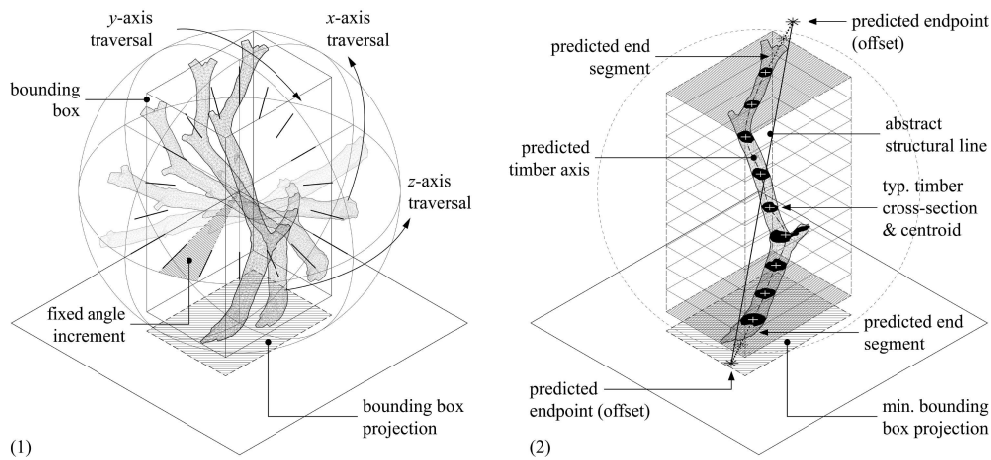


Fig. 2. (1) Posture Brute-Force Search Method, (2) Timber Axis and Virtual Structure Line Generation Process.

This process utilizes a script that sets the centroid of the initial posture model's bounding box as the base point. The model is then iteratively rotated by fixed angles along the x, y, and z axes to generate new bounding boxes. During this process, the projected area of each bounding box on the XY plane is monitored to identify the configuration with the smallest area. This method rapidly adjusts the initial model to the orientation closest to vertical based on the specified angle increments. Subsequently, the model is sliced according to the height (z-axis length) of the bounding box, with



slice density directly calculated by dividing this height. Listing 1 shows the highly simplified posture brute-force search script, using the projected area as the key value.

Listing 1. Highly Simplified Script for Posture Brute-Force Search Using Generators

```
def MinimumBoundingBox(geometry, angle_step):
    return min(
        (rotate(geometry, ax, ay, az) for ax, ay, az in angles),
        key=lambda g: compute_area(bounding_box(g))
    )
```

To mitigate the impact of the complex angles at the two end cross-sections of the timber on the accuracy of axis generation, this study calculates the tangents at the center points of the second and penultimate cross-sections. The intersections of these tangents with the top and bottom faces of the bounding box are extracted and offset outward by a fixed distance (to reserve space for nodes), serving as the start and end points of the axis. The axis is then constructed by interpolating a curve through the start point, the center points of the cross-sectional curves, and the end point. Meanwhile, the abstract structural line is directly formed by connecting the start and end points to facilitate subsequent structural generation operations.

2.3 Abstract Structure Generation Algorithm

Exploring how to automate the construction of 3D structures using abstract structural line sets is the core of this research and constitutes the second part of the overall workflow. The primary task of this stage is to perform mathematical operations on the length sets corresponding to the abstract structural lines generated in the first stage.

The tetrahedral framework is renowned in engineering and structural design for its exceptional stability and broad application value. As a fundamental unit of spatial trusses, a tetrahedron consists of four vertices and six edges. Its inherent self-constraining system allows forces to be transmitted along the edges to the vertices, effectively reducing local stress concentrations and exhibiting excellent rigidity [11]. Traditional spatial trusses are typically constructed by combining multiple standardized tetrahedral units. These units share edges on coincidence faces, enabling the construction of large-span and complex-shaped structures [11].

Figure 3 illustrates the logic of how this study utilizes recursive principles to construct irregular spatial structures based on a given set of random lengths.

The script first performs combination operations on the length set using the triangle inequality theorem to generate all qualifying base triangular frameworks. As shown in Figure 3 (2)-(8), for each base triangle, the script first extracts the plane's centroid C and offsets it along the negative z -axis by an arbitrary distance to obtain point C' . Subsequently, using `sympy.Eq()` and `sympy.Solve()`, the script permutes the base triangle's edge lengths with the remaining length set to solve a ternary quadratic equation for the fourth vertex. Among the two mirror solutions generated, the one farther from C' is selected by calculating their respective distances. This principle prevents unit

interleaving during recursion, thereby reducing the likelihood of structural generation failures.

After generating the first tetrahedral framework, the distance reference point shifts from C' to the centroid G of the tetrahedron. The script then uses the three newly generated faces as new base faces and repeats the aforementioned vertex-solving process. By iteratively updating the list of base triangles and using a `while` loop to progress further, the script continues generating new structures until the length set is exhausted or the recursion reaches a preset depth limit.

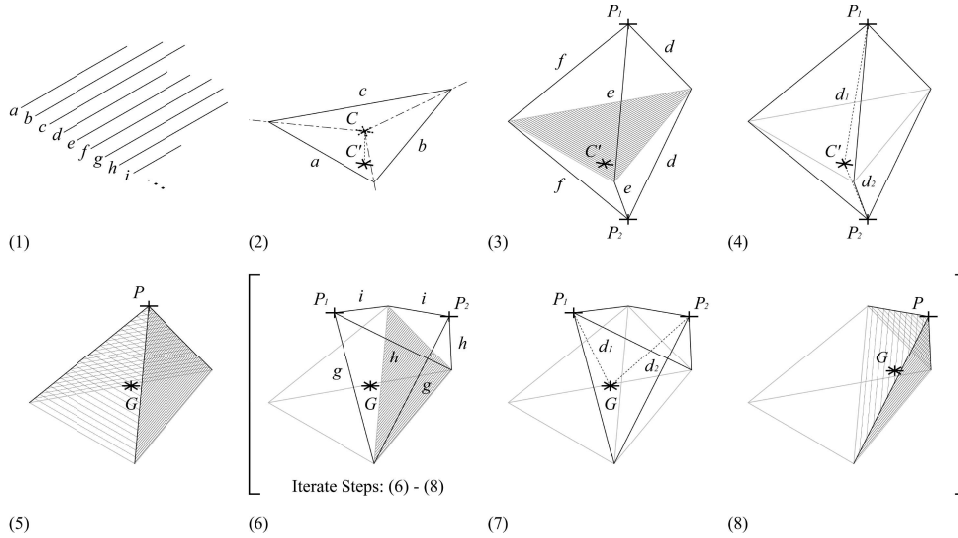


Fig. 3. Recursive Geometric Structure Generation with a Single Base Triangle

During testing, it was observed that when newly generated triangular faces are appended to the end of the list sequentially, the script completes all possibilities for the same vertex layer before moving to the next layer. While this approach covers all possibilities, the exponential growth of the dataset significantly increases the solving time, making it difficult for the script to quickly identify solutions with the maximum number of vertex layers. To address this, we employed the `insert()` method to insert the array of newly generated triangle edge lengths immediately after the current iteration object index. This ensures that each newly generated vertex prioritizes its associated triangular faces as base triangles for the next iteration rather than continuing to explore vertex combinations within the same layer. Testing showed that this method significantly accelerates the solving process aimed at maximizing the utilization of the length set, with efficiency improvements of up to several orders of magnitude. Figure 4 explains the logic of this optimization using simplified symbols.

The results generated through the above steps provide the 3D coordinates of each vertex in the composite structure. Further data processing produces a nested list, where each unit corresponds to the start and end coordinates of an abstract structural line. Since the computation time depends on the size of the original length set, to ensure users can obtain the best solution calculated before using a timer or manually



terminating the iteration, the results for vertices at each layer are stored incrementally in separate lists.

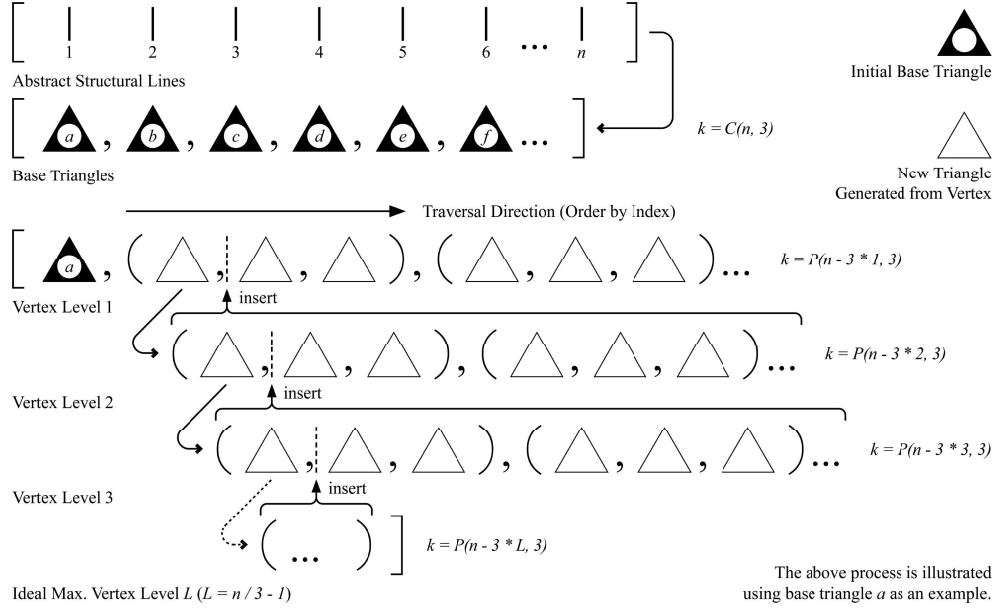


Fig. 4. Optimization Logic of the Vertex-Solving Algorithm: C for Combination, P for Permutation, k for Computational Complexity

The complete Python script for the above steps spans over 200 lines and encapsulates the process into a function with parameters for the length list, maximum layers, and time limit. Listing 2 presents the key logic of this process in the form of pseudocode.

Listing 2. Pseudocode for Recursive Vertex-Solving

```

Input: LengthList, LevelMaximum, TimeLimit
Output: BaseTriangles, VertexPoints, ConnectionLines
1. Initialize BaseTriangles, VertexPoints, and
   ConnectionLines as empty lists
2. Sort LengthList in descending order (optional)
3. Generate all combinations of 3 lengths from LengthList
   as BaseCombinations
4. Start Timer
5. For each Base in BaseCombinations:
   a. Check if Base forms a valid triangle using the
      triangle inequality
   b. If not valid, skip to the next Base
   c. Remove Base lengths from LengthList
   d. Initialize CurrentBase as the triangle formed by
      Base

```



```
6. While LengthList is not empty and Time elapsed <
TimeLimit:
    a. Generate all permutations of 3 lengths from
    LengthList as TripleCombinations
    b. For each Triple in TripleCombinations:
        i. Solve equations using SymPy to find possible
        vertex positions
            - Use the lengths in Triple and the points of
            CurrentBase
            - Solve a system of 3 quadratic equations
        ii. Select the vertex farthest from the CurrentBase
        centroid
        iii. If a valid vertex is found:
            - Add vertex to VertexPoints
            - Update CurrentBase and LengthList
            - Add new edges to ConnectionLines
            - Break to process the next vertex
7. Return BaseTriangles, VertexPoints, ConnectionLines
```

2.4 Visualization and Node Manufacturing Parameter Generation

The final stage of this process comprises three steps: visualization of the structural product, node generation, and the export of node manufacturing parameters. The primary objective is to create a complete product model, including node connection components, to facilitate visual inspection and provide precise parameters for subsequent node fabrication.

During the visualization phase, the Orient tool is employed to align and attach the processed timber models and their axes from Section 3.2 to the abstract structural framework generated in Section 3.3 by aligning their endpoints. Due to the rounding of results from previous SymPy calculations, the alignment tolerance is controlled within 2 millimeters. Subsequently, wooden spherical nodes of appropriate sizes are generated at each vertex, with dimensions determined based on the reserved space discussed in Section 3.2. Finally, wooden rods connecting the timber pieces and the spherical nodes are generated along the tangential directions at both endpoints of each axis. Figure 5 illustrates a result with two vertex layers.

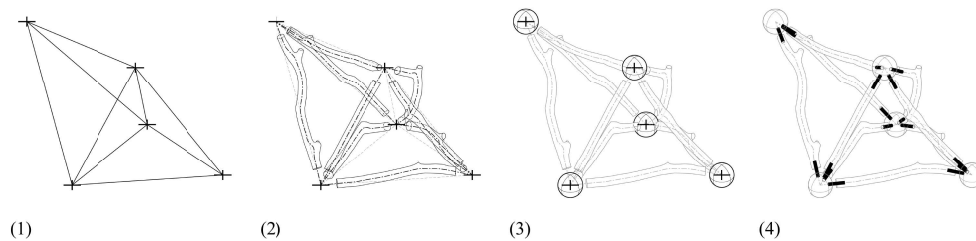


Fig. 5. (1) Vertices and Abstract Structural Lines, (2) Timber Model Alignment, (3) Spherical Node Generation, (4) Connecting Rod Generation.



In the context of predefined dimensions for connecting rods, the machining parameters in this study are defined as the drilling angles and depths for the timber and wooden spherical nodes. The drilling angle can be calculated using the unit vector of the tangent at the endpoint of the axis connected to the spherical node, with the sphere's center (vertex) as the origin. The drilling depth can be flexibly adjusted based on the standardized rod dimensions and the predefined reserved space.

2.5 FEA Parameter Configuration

We utilized ANSYS Workbench in the FEA process to perform stress tests on the generated structures, aiming to verify their reliability under practical load conditions.

To mitigate the potential impact of uncertainties, such as insect damage levels and localized structural defects, on the testing process, the wooden components in the FEA environment were modeled as isotropic materials with identical physical properties, as specified in Table 1. These parameter settings are based on pinewood, which is commonly used in building structures.

Table 1. Physical Parameter Settings Referencing Pinewood

Property	Value	Unit
Density	600.00	kg m ⁻³
Young's Modulus	1e+10	Pa
Poisson's Ratio	0.30	
Bulk Modulus	8.3333e+09	Pa
Shear Modulus	3.8462e+09	Pa
Tensile Yield Strength	4e+07	Pa
Compressive Ultimate Strength	6e+07	Pa

The above parameters were used to simulate stress distribution of structures under static loading conditions, either at a single node or multiple nodes. The load settings depend on the intended use of the product. According to the ISO-7173 standard, for structures potentially serving as landscape seats, a total load of 2000 (2*1000) N was applied during testing. For products intended to function as columns supporting flexible canopies and rigid wooden structures, the top node was estimated, based on prior physical constructions, to withstand a lateral force of no less than 400 N. The resulting bending moment was defined as $M = P \cdot d$, varying with the height of the structure.

3 Results

This chapter outlines the results corresponding to each step of the workflow described in Section 2.1. It covers the detailed characterization of test samples, the generation of spatial structures, finite element analysis using ANSYS Workbench, and an evaluation of the structures' application potential.

3.1 Results of Workflow-Controlled Structure and Parameter Generation

By applying the methods and processes described in Section 3, we tested 30 collected timber samples and successfully obtained the results. Figure 6 illustrates the dataset containing 30 sets of physical information processed through the workflow outlined in Section 3.2.

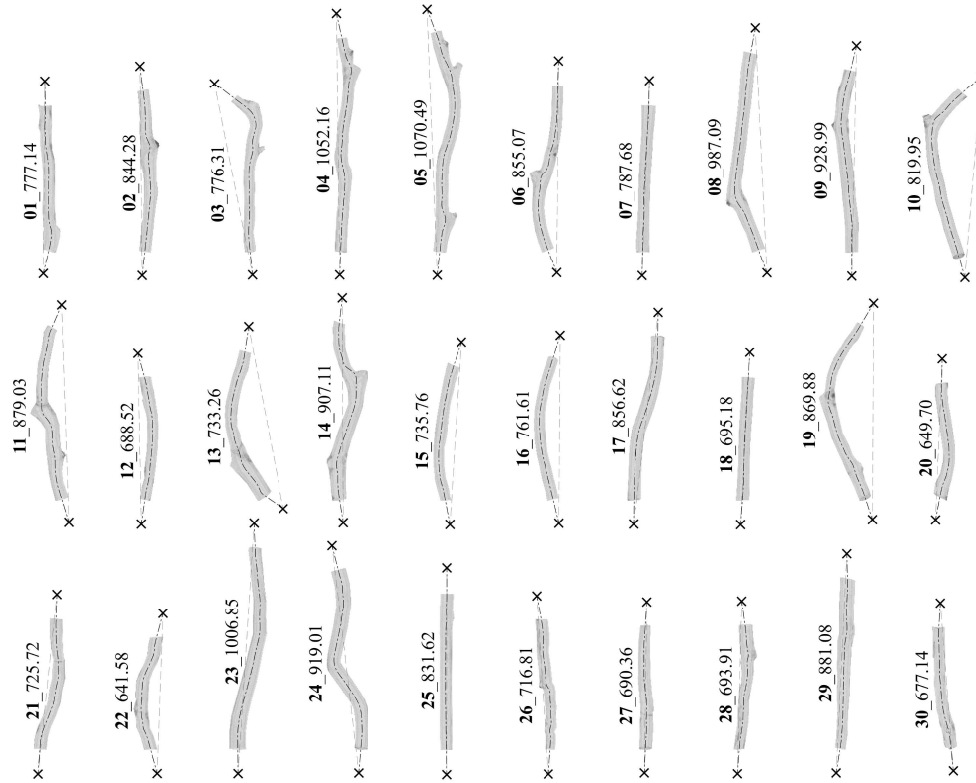


Fig. 6. Set of 30 Timber Axes, Endpoints, and Lengths in Translucent Mode (Unit: mm)

For efficiency and ease of demonstration, a script was used to randomly generate 21 unique integers within the range of 0–29, which were then used as indices to extract values from the length list for calculation. By performing computations on arrays generated with random seeds 1–10, 10 recursive structural frameworks were obtained. Figure 7 illustrates the output of the encapsulated function using random seed 1, displayed on the GHPython terminal. The data presents the virtual coordinates of each node relative to a fixed reference origin at (0, 0, 0). The coordinates are provided in millimeters, starting from the base triangular plane. With each newly identified apex of a tetrahedral unit during the search process, the corresponding coordinate is output to a new hierarchical level.

To enhance the diversity of generated forms, we bypassed the `sorted()` function in Step 2 of the pseudocode shown in Listing 2. Calculations were performed directly based on the combination order generated randomly. Figure 8 illustrates 10 structural combinations with low similarity, with timber numbers referenced from Figure 6.



```

----- LEVEL 001 -----
The Position of PeakBaseTriangle 1 : ((0, 0, 0), (762, 0, 0), (214, 642, 0))
The Position of Peak 1 : (390, 179, 597)
----- LEVEL 002 -----
The Position of PeakBaseTriangle 2 : ((762, 0, 0), (214, 642, 0), (390, 179, 597))
The Position of Peak 2 : (1105, 545, 242)
----- LEVEL 003 -----
The Position of PeakBaseTriangle 3 : ((214, 642, 0), (390, 179, 597), (1105, 545, 242))
The Position of Peak 3 : (314, 1044, 557)
----- LEVEL 004 -----
The Position of PeakBaseTriangle 4 : ((390, 179, 597), (1105, 545, 242), (314, 1044, 557))
The Position of Peak 4 : (904, 818, 842)
----- LEVEL 005 -----
The Position of PeakBaseTriangle 5 : ((1105, 545, 242), (314, 1044, 557), (904, 818, 842))
The Position of Peak 5 : (1051, 1315, 324)
----- LEVEL 006 -----
The Position of PeakBaseTriangle 6 : ((314, 1044, 557), (904, 818, 842), (1051, 1315, 324))
The Position of Peak 6 : (304, 1670, 697)
-----
Current a value : 0
Current b value : 5
Current c value : 0
GrandListTriangle Length Total : 28696
GrandListLevel Length Total : 28696
GrandListPeakUsed Length Total : 28696
Running Time Total : 436.08544874191284 sec
PROGRAM TERMINATION

Max Level For Each 'a' : [6]
Export Base Triangles : [(0, 0, 0), (762, 0, 0), (214, 642, 0)], [(762, 0, 0), (214, 642, 0), (390, 179, 597)], [(214, 642, 0), (390, 179, 597), (1105, 545, 242)],
[(390, 179, 597), (1105, 545, 242), (314, 1044, 557)], [(1105, 545, 242), (314, 1044, 557), (904, 818, 842)], [(314, 1044, 557), (904, 818, 842), (1051, 1315, 324)],
[(1051, 1315, 324), (304, 1670, 697)]
Export Peak Points : [(390, 179, 597), (1105, 545, 242), (314, 1044, 557), (904, 818, 842), (1051, 1315, 324), (304, 1670, 697)]
Export Lines : [ <System.Guid object at 0x00000289E9F92C00>, <System.Guid object at 0x00000289E9E8C900>, <System.Guid object at 0x00000289E9E8C2A0>, <System.Guid object at
0x00000289E9F6C600>, <System.Guid object at 0x00000289E9F9F3C0>, <System.Guid object at 0x00000289E9F8F580>, <System.Guid object at 0x00000289E9E9F900>, <System.Guid
object at 0x00000289E990A840>, <System.Guid object at 0x00000289E9B1D08800>, <System.Guid object at 0x00000289E9EDCE00>, <System.Guid object at 0x00000289E9B8F340>,
<System.Guid object at 0x00000289E9F8C200>, <System.Guid object at 0x00000289E9F2F400>, <System.Guid object at 0x00000289E9B4D0C0>, <System.Guid object at
0x00000289E9F3B140>, <System.Guid object at 0x00000289E9B04E800>, <System.Guid object at 0x00000289E9E28400>, <System.Guid object at 0x00000289E9025AC0>, <System.Guid
object at 0x00000289E984F92C0>, <System.Guid object at 0x00000289E9F16900>, <System.Guid object at 0x00000289E7C0A680>]

```

Fig. 7. Computation Results in GHPython Terminal

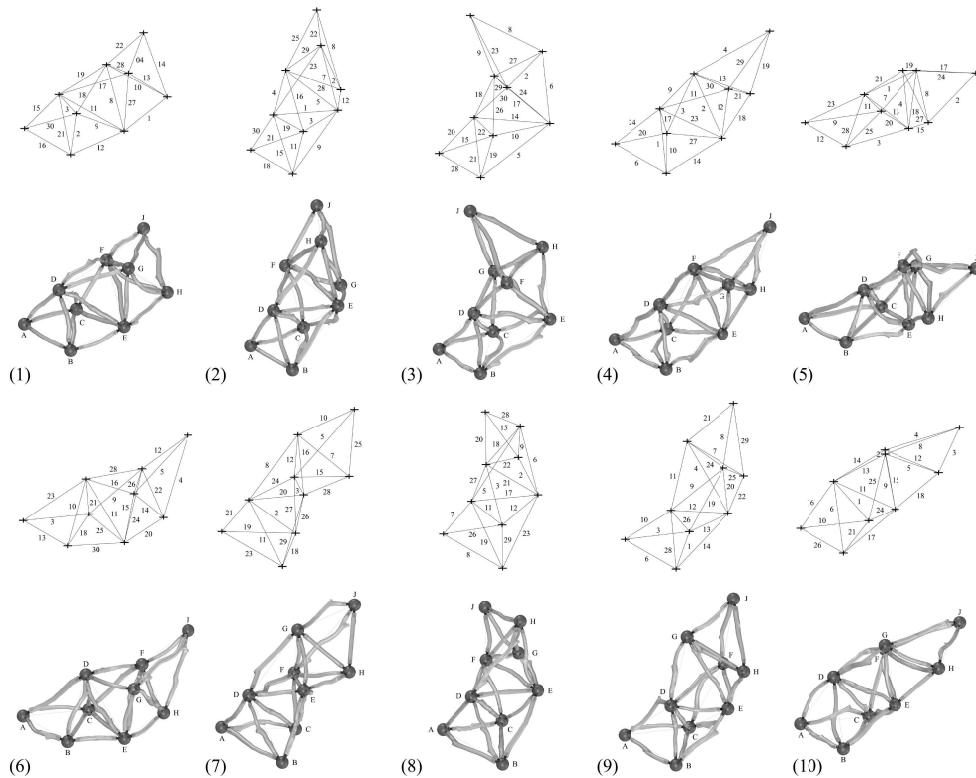


Fig. 8. 10 Results Generated from 21 Randomly Selected Lengths Out of 30

Subsequently, the machining parameters for each node were derived through straightforward data transformations. Figure 9 depicts the drilling start coordinates, depths, and diameters of wooden spherical nodes (150 mm diameter), with their centers serving as reference origins, based on the results generated using seed 1.



```

-A= [(21.396, 70.873, 12.010), (70.956, 19.385, 14.646), (25.086, 30.112, 63.945), 25.000, 30.000]
-B= [(47.450, 53.500, 22.609), (45.511, 59.011, 8.457), (41.195, 20.945, 59.070), (-72.421, -15.782, 9.930), 25.000, 30.000]
-C= [(-33.189, -67.295, -0.532), (43.871, -59.002, 14.402), (67.099, -22.879, 24.400), (12.614, 42.956, 40.122), (-5.312, -6.087, 74.812), 25.000, 30.000]
-D= [(-45.530, 5.434, -59.351), (42.740, -16.713, -59.321), (-29.515, 44.975, -52.260), (73.864, 11.975, -5.070), (38.834, 60.567, 21.180), (-14.640, 67.486, 29.263), 25.000, 30.000]
-E= [(-20.662, -67.776, -24.580), (-72.741, 6.988, -16.878), (-47.425, -40.191, 41.958), (-55.309, 49.363, 11.370), (-20.300, 32.919, 64.259), (10.993, 74.154, 2.310), 25.000, 30.000]
-F= [(-13.285, -44.538, -59.279), (22.956, -60.779, 37.469), (62.165, -19.490, -37.158), (53.731, 18.038, -49.118), (68.926, -14.479, 25.780), (-23.039, 71.147, 5.688), 25.000, 30.000]
-G= [(-39.621, -62.225, -13.537), (18.523, -30.363, -66.030), (4.535, 34.067, -66.662), (-62.522, 28.501, -30.063), (-39.102, 61.390, -17.918), 25.000, 30.000]
-H= [(6.886, -74.241, -8.113), (-70.299, 6.147, -25.403), (-24.656, -65.765, 26.308), (-58.539, 23.116, 19.824), 25.000, 30.000]
-I= [(-14.892, -67.002, -30.231), (52.625, -31.723, -43.003), (53.763, -48.626, 19.235), 25.000, 30.000]

```

Fig. 9. Machining Parameters for 9 Nodes Output by the GHPython Terminal

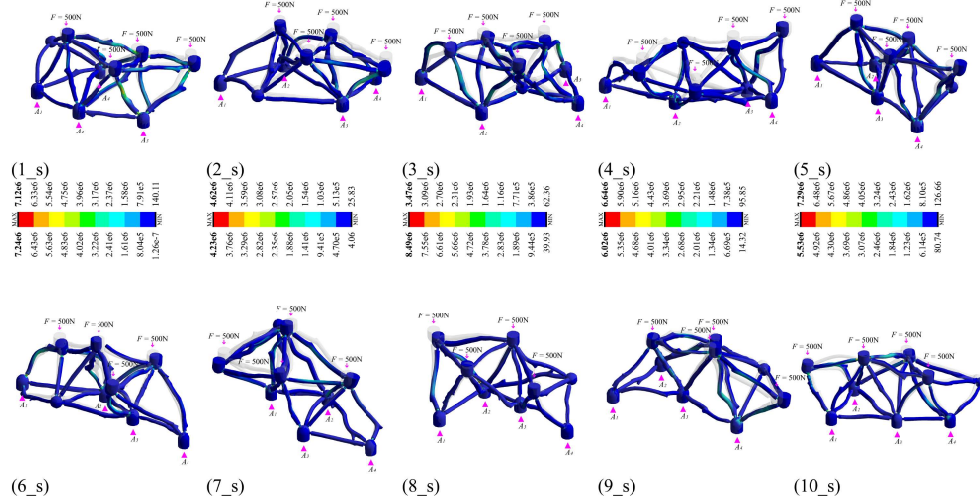
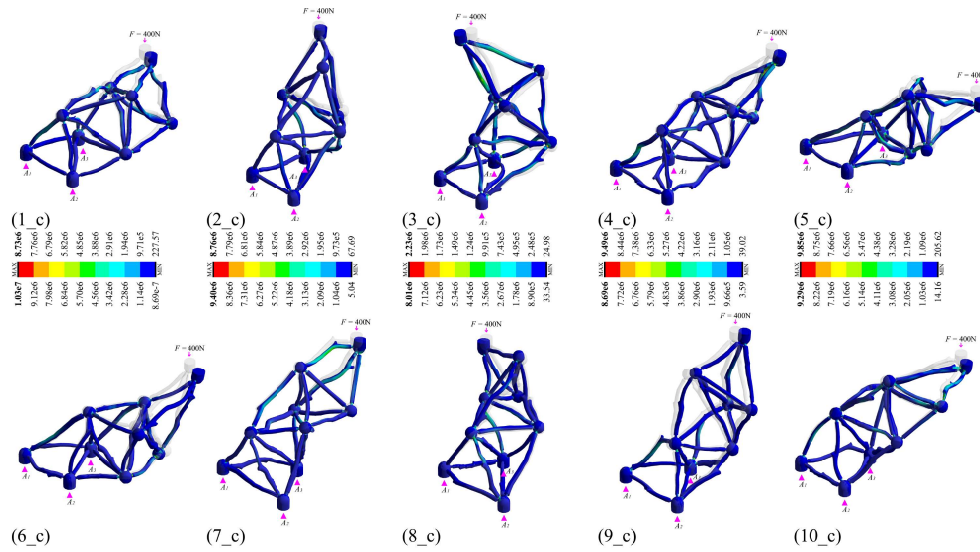


Fig. 10. Structural Equivalent Stress (Von-Mises) under Column (c) and Seat (s) Conditions

Figure 10 illustrates the equivalent stress distribution and deformation of the structure under two conditions: as a column support and as a landscape seat. In the column scenario (1_c–10_c), forces concentrate at the connections with the rigid canopy, whereas in the seat scenario (1_s–10_s), loads are transferred from the upper nodes to



the fixed supports at the base. The maximum stress values suggest that the structures remain stable under the given loads.

Overall, the generated structures can support canopy systems woven from wood shavings, as shown in Figure 11(1), derived from our group's prior work. They can also be adapted into functional furniture by integrating rigid panels, as demonstrated in the design scheme in Figure 11(2), using conventional fabrication techniques.

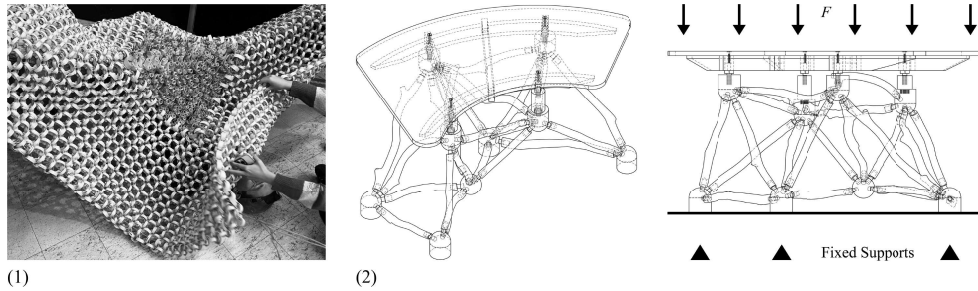


Fig. 11. (1) Woven Wood Shavings Canopy – Previous Work by the Team, (2) Seating Design with a Single Rigid Panel Applied to Result 3_s.

4 Limitations

The current script generates structures in random directions, limiting suitability for specific sites. Filtering by tetrahedral alignment is possible but computationally intensive. Without physical testing, the end-effector's ability to drill and assemble accurately remains unverified.

5 Conclusion

This study presents a computational scripting-based method for transforming curved wood waste and by-products from commercial logging processes into structural components with both economic and aesthetic value. Building on prior research conducted at the MAAD Studio One at UC Berkeley, the project aims to provide a structurally stable, automation-compatible support system for woven canopies made from elastic wood by-products—addressing the instability and randomness caused by manual installation.

A structure generator was developed and tested using GHPython, combining a brute-force 3D posture search with fixed angular increments and structural combination solving via SymPy. This approach enables users to rapidly generate structural solutions and corresponding fabrication parameters based on scanned geometries of valid timber pieces. FEA was subsequently used to evaluate structural performance under realistic load conditions.

The results, as illustrated in Figures 8 and 10, validate both the feasibility of the generator and the mechanical stability of the generated structures.



This work explores the forward-looking potential of bridging natural material behavior with computational design. A key appeal lies in how a logic-driven script, combined with tools such as depth cameras, scanners, or sensors, can enable robots to recognize and fabricate components with human-like flexibility. The developed scripting system shows strong potential for integration with current robot and cobot platforms. By automating the transformation of timber waste into environmentally responsible structural products, the method enhances both economic efficiency and carbon reduction, while introducing novel aesthetic and spatial possibilities in architectural design.

Future research will focus on improving form controllability and optimizing algorithmic efficiency. We are optimistic that this approach can contribute meaningfully to sustainable construction and enhanced spatial quality in the built environment.

Acknowledgments. The author gratefully acknowledges the mentorship and intellectual guidance of Professor Maria Paz Gutierrez. Special thanks are also due to the MAAD 2023 cohort, whose collective engagement and contributions greatly enriched the research process.

References

1. Connick, J., Rogers, L., Wheeler, K.: Increasing Mass Timber Consumption in the U.S. and Sustainable Timber Supply. *Sustainability*. 14, 381 (2021)
2. Alvarez, M.: *The State of America's Forests*. Society of American Foresters, Bethesda, MD (2007)
3. Đuka, A., Sertić, M., Pentek, T., Papa, I., Janeš, D., Poršinsky, T.: Round Wood Waste and Losses: Is Rationalisation in Scaling Possible? *Croat. j. for. eng. (Online)*. 41, 287–298 (2020)
4. Pramreiter, M., Nenning, T., Huber, C., Müller, U., Kromoser, B., Mayencourt, P., Konnerth, J.: A review of the resource efficiency and mechanical performance of commercial wood-based building materials. *Sustainable Materials and Technologies*. 38, e00728 (2023)
5. Krishna, M.P., Mohan, M.: Litter decomposition in forest ecosystems: a review. *Energ. Ecol. Environ.* 2, 236–249 (2017)
6. Barham, L., Duller, G.A.T., Candy, I., Scott, C., Cartwright, C.R., Peterson, J.R., Kabukcu, C., Chapot, M.S., Melia, F., Rots, V., George, N., Taipale, N., Gethin, P., Nkombwe, P.: Evidence for the earliest structural use of wood at least 476,000 years ago. *Nature*. 622, 107–111 (2023)
7. Monier, V., Bignon, J.C., Duchanois, G.: Use of Irregular Wood Components to Design Non-Standard Structures. *AMR*. 671–674, 2337–2343 (2013)
8. Devadass, P., Dailami, F., Mollica, Z., Self, M.: *Robotic Fabrication of Non-Standard Material*. Presented at the ACADIA 2016: Post-Human Frontiers, Ann Arbor (Michigan), USA (2016)
9. Reisach, D., Schütz, S., Willmann, J., Schneider, S.: A Design-to-Fabrication Workflow for Free-Form Timber Structures Using Offcuts. In: Turrin, M., Andriotis, C., and Rafiee, A. (eds.) *Computer-Aided Architectural Design. INTERCONNECTIONS: Co-computing Beyond Boundaries*. pp. 361–375. Springer Nature Switzerland, Cham (2023)
10. Wang, Y.: Hooke Park Biomass Boiler House. In: *Advancing Wood Architecture*, pp. 169–181. Architectural Association School of Architecture, England (2007)
11. Lan, T.T.: *Space Frame Structures*. In: Chen, W.F. (ed.) *Structural Engineering Handbook*. CRC Press LLC, Boca Raton (1999)

